DATA 412/612 · Statistical Programming in R · Week 14

Functional Programming with purrr: 20 Examples

Twenty examples from map() to walk2(), each with a worked example and a solution. Examples 11–20 keep the solution closed until you expand it.

Apply the same operation to each piece: a vector, a list, the columns of a data frame, a group, or several inputs side by side.

Datasets: `iris`, `mtcars`, `airquality`, `ToothGrowth`, `ChickWeight`.

Run the worked example. Then write Your turn before you read the solution. Some exercises use different columns from the worked example. Examples 11--20 continue the same pattern, and each solution stays closed until you expand it.

```
set.seed(612)
library(tidyverse)
```

---

1. A list and `map()`

`mtcars`. Split a column, then apply one function to each piece.

Worked example

```
mpg_by_cyl <- split(mtcars$mpg, mtcars$cyl)
str(mpg_by_cyl)
```

```
## List of 3
##  $ 4: num [1:11] 22.8 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26 30.4 ...
##  $ 6: num [1:7] 21 21 21.4 18.1 19.2 17.8 19.7
##  $ 8: num [1:14] 18.7 14.3 16.4 17.3 15.2 10.4 10.4 14.7 15.5 15.2 ...
```

```
map(mpg_by_cyl, mean)
```

```
## $`4`
## [1] 26.66364
##
## $`6`
## [1] 19.74286
##
## $`8`
## [1] 15.1
```

```
map_dbl(mpg_by_cyl, mean)
```

```
##        4        6        8
## 26.66364 19.74286 15.10000
```

`map()` applies `mean` to the 4-, 6-, and 8-cylinder groups. Pass `mean` without parentheses.

Your turn

Split `mtcars$hp` by `mtcars$gear`. Map `max`. Repeat with `map_dbl()`. What changes?

Solution

```
hp_by_gear <- split(mtcars$hp, mtcars$gear)
map(hp_by_gear, max)
```

```
## $`3`
## [1] 245
##
## $`4`
## [1] 123
##
## $`5`
## [1] 335
```

```
map_dbl(hp_by_gear, max)
```

```
##   3   4   5
## 245 123 335
```

`map()` returns a list. `map_dbl()` returns a double vector. Each group must produce one numeric value.

---

2. `map()` on data-frame columns

`iris`. A data frame is a list of columns, so `map()` walks the columns.

Worked example

```
map(iris, class)
```

```
## $Sepal.Length
## [1] "numeric"
##
## $Sepal.Width
## [1] "numeric"
##
## $Petal.Length
## [1] "numeric"
##
## $Petal.Width
## [1] "numeric"
##
## $Species
## [1] "factor"
```

```
iris |>
  select(where(is.numeric)) |>
  map_dbl(sd)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##    0.8280661    0.4358663    1.7652982    0.7622377
```

`Species` is a factor. Select the numeric columns before `sd()`.

Your turn

Median of every numeric `iris` column, as one named numeric vector. Class of each column, as a character vector. Why does `map_chr(iris, class)` work here?

Solution

```
iris |>
  select(where(is.numeric)) |>
  map_dbl(median)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width
##         5.80         3.00         4.35         1.30
```

```
map_chr(iris, class)
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##    "numeric"    "numeric"    "numeric"    "numeric"     "factor"
```

`class()` produces one string per column here. `map_chr()` requires exactly one character value per element.

---

3. Missing values and a lambda

`airquality`. `\(x)` is short for `function(x)`, from R 4.1.

Worked example

```
map_lgl(airquality, \(x) any(is.na(x)))
```

```
##   Ozone Solar.R    Wind    Temp   Month     Day
##    TRUE    TRUE   FALSE   FALSE   FALSE   FALSE
```

```
map_int(airquality, \(x) sum(is.na(x)))
```

```
##   Ozone Solar.R    Wind    Temp   Month     Day
##      37       7       0       0       0       0
```

The first call is logical. The second is an integer count.

Your turn

Proportion missing in every column, with `map_dbl()`. Names of the columns that have any missing value. Mean of `Ozone`, ignoring missing values.

Solution

```
missing_proportion <- map_dbl(airquality, \(x) mean(is.na(x)))
missing_proportion
```

```
##      Ozone    Solar.R       Wind       Temp      Month        Day
## 0.24183007 0.04575163 0.00000000 0.00000000 0.00000000 0.00000000
```

```
missing_any <- map_lgl(airquality, \(x) any(is.na(x)))
names(missing_any)[missing_any]
```

```
## [1] "Ozone"   "Solar.R"
```

```
mean(airquality$Ozone, na.rm = TRUE)
```

```
## [1] 42.12931
```

```
map_dbl(airquality["Ozone"], \(x) mean(x, na.rm = TRUE))
```

```
##    Ozone
## 42.12931
```

`mean(is.na(x))` is the fraction of `TRUE`, because `TRUE` is 1 and `FALSE` is 0.

---

4. `across()`

`iris`. `across()` inside `summarise()` keeps one row per species. `map()` on a data frame does not keep the grouping column.

Worked example

```
iris |>
  group_by(Species) |>
  summarise(
    across(
      ends_with("Length"),
      list(mean = mean, sd = sd),
      .names = "{.col}_{.fn}"
    ),
    .groups = "drop"
  )
```

```
## # A tibble: 3 × 5
##   Species    Sepal.Length_mean Sepal.Length_sd Petal.Length_mean Petal.Length_sd
##   <fct>                  <dbl>           <dbl>             <dbl>           <dbl>
## 1 setosa                  5.01           0.352              1.46           0.174
## 2 versicolor              5.94           0.516              4.26           0.470
## 3 virginica               6.59           0.636              5.55           0.552
```

Your turn

For each species, the median and the IQR of `Sepal.Width` and `Petal.Width`. Names such as `Sepal.Width_median` and `Sepal.Width_IQR`.

Solution

```
iris |>
  group_by(Species) |>
  summarise(
    across(
      c(Sepal.Width, Petal.Width),
      list(median = median, IQR = IQR),
      .names = "{.col}_{.fn}"
    ),
    .groups = "drop"
  )
```

```
## # A tibble: 3 × 5
##   Species  Sepal.Width_median Sepal.Width_IQR Petal.Width_median Petal.Width_IQR
##   <fct>                 <dbl>           <dbl>              <dbl>           <dbl>
## 1 setosa                  3.4           0.475                0.2             0.1
## 2 versico…                2.8           0.475                1.3             0.3
## 3 virgini…                3             0.375                2               0.5
```

One row for each of the three species.

---

5. One function, many groups

`ToothGrowth`. A named function can return a one-row data frame. `map()` then returns a list of those data frames.

`map_dfr()` and `map_dfc()` still run and purrr marks them superseded; collect data frames with `map(...) |> list_rbind()` or `list_cbind()`.

Worked example

```
mean_sd <- function(x) {
  tibble(mean = mean(x, na.rm = TRUE), sd = sd(x, na.rm = TRUE))
}
length_by_supp <- split(ToothGrowth$len, ToothGrowth$supp)
results <- map(length_by_supp, mean_sd)
results
```

```
## $OJ
## # A tibble: 1 × 2
##    mean    sd
##   <dbl> <dbl>
## 1  20.7  6.61
##
## $VC
## # A tibble: 1 × 2
##    mean    sd
##   <dbl> <dbl>
## 1  17.0  8.27
```

```
bind_rows(results, .id = "supp")
```

```
## # A tibble: 2 × 3
##   supp   mean    sd
##   <chr> <dbl> <dbl>
## 1 OJ     20.7  6.61
## 2 VC     17.0  8.27
```

`.id` puts the list names `OJ` and `VC` into `supp`.

Your turn

Write `five_number()` so it returns a one-row tibble with the minimum, the median, and the maximum. Apply it to `ToothGrowth$len` split by `dose`. Combine the rows and keep a `dose` column.

Solution

```
five_number <- function(x) {
  tibble(minimum = min(x), median = median(x), maximum = max(x))
}
ToothGrowth$len |>
  split(ToothGrowth$dose) |>
  map(five_number) |>
  bind_rows(.id = "dose")
```

```
## # A tibble: 3 × 4
##   dose  minimum median maximum
##   <chr>   <dbl>  <dbl>   <dbl>
## 1 0.5       4.2   9.85    21.5
## 2 1        13.6  19.2     27.3
## 3 2        18.5  26.0     33.9
```

`dose` is character because it was a list name. Use `as.numeric(dose)` when you need a number.

---

6. `map2()`

`mtcars`. Two vectors, paired by position.

Worked example

```
mpg <- mtcars$mpg[1:5]
distance <- c(100, 150, 80, 120, 200)
price_per_gallon <- 3.50
cost <- map2_dbl(distance, mpg, \(d, m) (d / m) * price_per_gallon)
tibble(car = rownames(mtcars)[1:5], distance, mpg, cost)
```

```
## # A tibble: 5 × 4
##   car               distance   mpg  cost
##   <chr>                <dbl> <dbl> <dbl>
## 1 Mazda RX4              100  21    16.7
## 2 Mazda RX4 Wag          150  21    25
## 3 Datsun 710              80  22.8  12.3
## 4 Hornet 4 Drive         120  21.4  19.6
## 5 Hornet Sportabout      200  18.7  37.4
```

Your turn

The first five cars. Distances `c(50, 75, 100, 125, 150)`. Four dollars a gallon. `map2_dbl()` for the fuel cost. A tibble with the car name and the fuel cost.

Solution

```
d <- c(50, 75, 100, 125, 150)
m <- mtcars$mpg[1:5]
fuel_cost <- map2_dbl(d, m, \(distance, mpg) distance / mpg * 4)
tibble(car = rownames(mtcars)[1:5], fuel_cost)
```

```
## # A tibble: 5 × 2
##   car               fuel_cost
##   <chr>                 <dbl>
## 1 Mazda RX4              9.52
## 2 Mazda RX4 Wag         14.3
## 3 Datsun 710            17.5
## 4 Hornet 4 Drive        23.4
## 5 Hornet Sportabout     32.1
```

---

7. `pmap()`

`mtcars`. Three inputs, still paired by position.

Worked example

```
trips <- tibble(
  car = rownames(mtcars)[1:4],
  distance = c(50, 100, 150, 200),
  mpg = mtcars$mpg[1:4],
  price = c(3.25, 3.50, 3.75, 4.00)
)
trips |>
  mutate(
    cost = pmap_dbl(
      list(distance, mpg, price),
      \(distance, mpg, price) distance / mpg * price
    )
  )
```

```
## # A tibble: 4 × 5
##   car            distance   mpg price  cost
##   <chr>             <dbl> <dbl> <dbl> <dbl>
## 1 Mazda RX4            50  21    3.25  7.74
## 2 Mazda RX4 Wag       100  21    3.5  16.7
## 3 Datsun 710          150  22.8  3.75 24.7
## 4 Hornet 4 Drive      200  21.4  4    37.4
```

Your turn

A shipping tibble: `weight = c(2, 5, 8)`, `rate = c(1.5, 2, 2.5)`, `base_fee = c(3, 4, 5)`. `pmap_dbl()` for `base_fee + weight * rate`.

Solution

```
shipping <- tibble(
  weight = c(2, 5, 8),
  rate = c(1.5, 2, 2.5),
  base_fee = c(3, 4, 5)
)
shipping |>
  mutate(
    cost = pmap_dbl(
      list(base_fee, weight, rate),
      \(base_fee, weight, rate) base_fee + weight * rate
    )
  )
```

```
## # A tibble: 3 × 4
##   weight  rate base_fee  cost
##    <dbl> <dbl>    <dbl> <dbl>
## 1      2   1.5        3     6
## 2      5   2          4    14
## 3      8   2.5        5    25
```

The costs are 6, 14, and 25.

---

8. `group_map()` and `group_modify()`

`ChickWeight`. `data` is the group's rows. `key` is the group id. `group_map()` returns a list. `group_modify()` binds the data frames and keeps the group column.

Worked example

```
fit_slope <- function(data) {
  fit <- lm(weight ~ Time, data = data)
  tibble(slope = unname(coef(fit)["Time"]))
}
ChickWeight |>
  group_by(Diet) |>
  group_map(\(data, key) fit_slope(data))
```

```
## [[1]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  6.84
##
## [[2]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  8.61
##
## [[3]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  11.4
##
## [[4]]
## # A tibble: 1 × 1
##   slope
##   <dbl>
## 1  9.71
```

```
ChickWeight |>
  group_by(Diet) |>
  group_modify(\(data, key) fit_slope(data)) |>
  ungroup()
```

```
## # A tibble: 4 × 2
##   Diet  slope
##   <fct> <dbl>
## 1 1      6.84
## 2 2      8.61
## 3 3     11.4
## 4 4      9.71
```

Your turn

`mtcars`. Fit `mpg ~ wt` inside each `cyl` group. One row per cylinder group, with the slope on `wt`. Which group has the most negative slope?

Solution

```
model_slopes <- mtcars |>
  group_by(cyl) |>
  group_modify(\(data, key) {
    model <- lm(mpg ~ wt, data = data)
    tibble(wt_slope = unname(coef(model)["wt"]))
  }) |>
  ungroup()
model_slopes
```

```
## # A tibble: 3 × 2
##     cyl wt_slope
##   <dbl>    <dbl>
## 1     4    -5.65
## 2     6    -2.78
## 3     8    -2.19
```

```
model_slopes |>
  slice_min(wt_slope, n = 1)
```

```
## # A tibble: 1 × 2
##     cyl wt_slope
##   <dbl>    <dbl>
## 1     4    -5.65
```

A more negative slope means fitted MPG falls faster as weight rises inside that cylinder group. That is an association, not a cause.

9. `imap()`

`iris`. `imap()` passes the element and its name.

Worked example

```
numeric_iris <- iris |>
  select(where(is.numeric))
column_reports <- imap_chr(
  numeric_iris,
  \(values, column_name) paste0(column_name, ": mean = ", round(mean(values), 2))
)
column_reports
```

```
##                Sepal.Length                 Sepal.Width
## "Sepal.Length: mean = 5.84"  "Sepal.Width: mean = 3.06"
##                Petal.Length                 Petal.Width
## "Petal.Length: mean = 3.76"   "Petal.Width: mean = 1.2"
```

Your turn

`imap_chr()` on the numeric columns of `mtcars`. Strings like `mpg: min = 10.4`.

Solution

```
mtcars |>
  select(where(is.numeric)) |>
  imap_chr(\(values, column_name) paste0(column_name, ": min = ", min(values)))
```

```
##                mpg                cyl               disp                 hp
##  "mpg: min = 10.4"     "cyl: min = 4" "disp: min = 71.1"     "hp: min = 52"
##               drat                 wt               qsec                 vs
## "drat: min = 2.76"  "wt: min = 1.513" "qsec: min = 14.5"      "vs: min = 0"
##                 am               gear               carb
##      "am: min = 0"    "gear: min = 3"    "carb: min = 1"
```

---

10. `walk()` for a side effect

`iris`. `walk()` is for a file or a plot. It returns the input invisibly. The files below are written under `tempdir()`, not in the project folder.

Worked example

```
plot_dir <- file.path(tempdir(), "data612_iris_plots")
unlink(plot_dir, recursive = TRUE)
dir.create(plot_dir)
plot_vars <- c("Sepal.Length", "Sepal.Width", "Petal.Length")
walk(plot_vars, \(var) {
  p <- ggplot(iris, aes(x = .data[[var]])) +
    geom_histogram(bins = 15)
  ggsave(
    filename = file.path(plot_dir, paste0(var, ".png")),
    plot = p,
    width = 5,
    height = 3.5,
    units = "in"
  )
})
list.files(plot_dir)
```

```
## [1] "Petal.Length.png" "Sepal.Length.png" "Sepal.Width.png"
```

Your turn

Four histogram PNGs, one per numeric `iris` column, in a different temporary directory. Check that four PNG files are there.

Solution

```
out_dir <- file.path(tempdir(), "data612_iris_numeric")
unlink(out_dir, recursive = TRUE)
dir.create(out_dir)
numeric_cols <- names(iris)[map_lgl(iris, is.numeric)]
walk(numeric_cols, \(var) {
  p <- ggplot(iris, aes(x = .data[[var]])) +
    geom_histogram(bins = 15)
  ggsave(
    filename = file.path(out_dir, paste0(var, ".png")),
    plot = p,
    width = 5,
    height = 3.5,
    units = "in"
  )
})
saved_files <- list.files(out_dir)
saved_files
```

```
## [1] "Petal.Length.png" "Petal.Width.png"  "Sepal.Length.png" "Sepal.Width.png"
```

```
length(saved_files)
```

```
## [1] 4
```

Quick reference

Goal Function Output

Apply one function to `map()` List each element

Return one number per `map_dbl()` Double vector element

Return one integer per `map_int()` Integer vector element

Return one string per `map_chr()` Character vector element

Return one `map_lgl()` Logical vector `TRUE` / `FALSE` per element

Apply functions across `across()` Depends on `mutate()` / selected columns `summarise()`

Iterate over two inputs `map2()` / `map2_dbl()` List / double vector

Iterate over many `pmap()` / `pmap_dbl()` List / double vector inputs

Work with elements and `imap()` / `imap_chr()` List / character vector their names

Return a list of `group_map()` List grouped results

Combine data frames `group_modify()` Data frame from groups

Write files or generate `walk()` The input, invisibly plots

Concept check

Why is `map_dbl()` often preferable to `map()` when every iteration must produce a single number?

Why is `walk()` more appropriate than `map()` when saving images?

Examples 11--20

These are **10 additional examples** that build on examples 1--10. Each example has (i) a question to discuss, (ii) a guided demonstration, (iii) an exercise, and (iv) a solution you can expand after trying the

exercise. The examples use only datasets distributed with R: `mtcars`, `iris`, `airquality`, and `ToothGrowth`.

Key idea: The first argument of `split(x, f)` is **what you want to divide**. If `x` is a vector, you get a list of vectors; if `x` is a data frame, you get a list of data frames. The second argument `f` determines the groups.

Example 11 --- Split a vector by month, then calculate summaries

Dataset: `airquality` (daily air-quality measurements in New York).

Goal: Distinguish *grouping values* from *calculating within each group*.

Worked example

```
# Split the temperature vector into groups defined by Month.
temp_by_month <- split(airquality$Temp, airquality$Month)

names(temp_by_month)  # Group labels
```

```
## [1] "5" "6" "7" "8" "9"
```

```
lengths(temp_by_month) # Days in each month
```

```
##  5  6  7  8  9
## 31 30 31 31 30
```

```
str(temp_by_month)    # A list of numeric vectors
```

```
## List of 5
##  $ 5: int [1:31] 67 72 74 62 56 66 65 59 61 69 ...
##  $ 6: int [1:30] 78 74 67 84 85 79 82 87 90 87 ...
##  $ 7: int [1:31] 84 85 81 84 83 83 88 92 92 89 ...
##  $ 8: int [1:31] 81 81 82 86 85 87 89 90 90 92 ...
##  $ 9: int [1:30] 91 92 93 93 87 84 80 78 75 73 ...
```

```
map(temp_by_month, max)       # List output
```

```
## $`5`
## [1] 81
##
## $`6`
## [1] 93
##
## $`7`
## [1] 92
##
## $`8`
## [1] 97
##
## $`9`
## [1] 93
```

```
map_dbl(temp_by_month, max)   # Named double vector
```

```
##  5  6  7  8  9
## 81 93 92 97 93
```

```
map_dbl(temp_by_month, mean)  # Average temperature by month
```

```
##        5        6        7        8        9
## 65.54839 79.10000 83.90323 83.96774 76.90000
```

Discuss: `split()` has not computed any averages. It has only organized values into separate vectors. The calculation happens in `map()`.

Your turn

1. Split **wind speed** by month.
2. Find the minimum wind speed for each month with `map_dbl()`.
3. Find the difference between the maximum and minimum wind speed in each month. Use a lambda function.

Solution — Example 11

```
wind_by_month <- split(airquality$Wind, airquality$Month)
map_dbl(wind_by_month, min)
```

```
##   5   6   7   8   9
## 5.7 1.7 4.1 2.3 2.8
```

```
map_dbl(wind_by_month, \(x) max(x) - min(x))
```

```
##    5    6    7    8    9
## 14.4 19.0 10.8 13.2 13.8
```

Takeaway: A lambda such as `\(x) max(x) - min(x)` is helpful when the operation is more complicated than a single existing function.

Example 12 --- Split a complete data frame and keep all its columns

Dataset: `iris`.

Goal: See why `split(iris, iris$Species)` is different from `split(iris$Sepal.Length, iris$Species)`.

Worked example

```
iris_groups <- split(iris, iris$Species)

class(iris_groups)             # list
```

```
## [1] "list"
```

```
map_int(iris_groups, nrow)     # rows per species
```

```
##     setosa versicolor  virginica
##         50         50         50
```

```
map_int(iris_groups, ncol)     # columns per species
```

```
##     setosa versicolor  virginica
##          5          5          5
```

```
names(iris_groups)            # species labels
```

```
## [1] "setosa"     "versicolor" "virginica"
```

```
# Each element is a complete data frame, so we can use multiple columns.
map_dbl(iris_groups, \(d) cor(d$Sepal.Length, d$Petal.Length))
```

```
##     setosa versicolor  virginica
##  0.2671758  0.7540490  0.8642247
```

Discuss: Splitting the complete `iris` object preserves all five columns in each group. Splitting `iris$Sepal.Length` preserves only that numerical variable.

Your turn

1. Split the **entire** `mtcars` data frame by `cyl`.
2. Find the number of cars per cylinder group.
3. Find the mean `mpg` within each group, using `map_dbl()`.

Solution — Example 12

```
cars_by_cyl <- split(mtcars, mtcars$cyl)
map_int(cars_by_cyl, nrow)
```

```
##  4  6  8
## 11  7 14
```

```
map_dbl(cars_by_cyl, \(d) mean(d$mpg))
```

```
##        4        6        8
## 26.66364 19.74286 15.10000
```

Takeaway: When a list element is a data frame, the anonymous function receives a data frame (`d`) rather than a single number.

Example 13 --- Handle missing values inside mapping functions

Dataset: `airquality`.

Goal: Learn why `na.rm = TRUE` matters when summarizing columns.

Worked example

```
# Airquality has missing values in some columns.
map_int(airquality, \(x) sum(is.na(x)))
```

```
##   Ozone Solar.R    Wind    Temp   Month     Day
##      37       7       0       0       0       0
```

```
# Without na.rm, the Ozone average is missing.
mean(airquality$Ozone)
```

```
## [1] NA
```

```
mean(airquality$Ozone, na.rm = TRUE)
```

```
## [1] 42.12931
```

```
# Compute means for every column safely.
map_dbl(airquality, \(x) mean(x, na.rm = TRUE))
```

```
##      Ozone    Solar.R       Wind       Temp      Month        Day
##  42.129310 185.931507   9.957516  77.882353   6.993464  15.803922
```

Discuss: `map_dbl()` expects **one number per column**, but that number can be `NA_real_` if the chosen function produces a missing result. It does not automatically remove missing observations.

Your turn

1. Calculate the **median** of every column in `airquality`, ignoring missing observations.
2. Calculate the 90th percentile of each column, ignoring missing observations.
3. Why would `map_dbl(airquality, quantile)` fail with default arguments?

Solution — Example 13

```
map_dbl(airquality, \(x) median(x, na.rm = TRUE))
```

```
##   Ozone Solar.R    Wind    Temp   Month     Day
##    31.5   205.0     9.7    79.0     7.0    16.0
```

```
map_dbl(airquality, \(x) unname(quantile(x, probs = 0.90, na.rm = TRUE)))
```

```
##   Ozone Solar.R    Wind    Temp   Month     Day
##    87.0   288.5    14.9    90.0     9.0    28.0
```

`quantile(x)` usually returns **five values** (0%, 25%, 50%, 75%, 100%), not one. `map_dbl()` requires a single double value for each column; the default `quantile()` call may also encounter missing values.

Example 14 --- Use `map_lgl()` to check properties of each column

Dataset: `iris`.

Goal: Write functions that return `TRUE` or `FALSE` for each column.

Worked example

```
map_lgl(iris, is.numeric)              # Which columns are numeric?
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##         TRUE         TRUE         TRUE         TRUE        FALSE
```

```
map_lgl(iris, is.factor)               # Which columns are factors?
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##        FALSE        FALSE        FALSE        FALSE         TRUE
```

```
map_lgl(iris, \(x) any(is.na(x)))      # Any missing values?
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##        FALSE        FALSE        FALSE        FALSE        FALSE
```

```
map_lgl(iris, \(x) length(unique(x)) > 10) # More than 10 distinct values?
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##         TRUE         TRUE         TRUE         TRUE        FALSE
```

Discuss: `map_lgl()` does not keep a list of TRUE/FALSE values; it returns a named logical vector.

Your turn

For `mtcars`, use `map_lgl()` to find which columns have (a) at least 10 distinct values and (b) any value greater than 200.

Solution — Example 14

```
map_lgl(mtcars, \(x) length(unique(x)) >= 10)
```

```
##   mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
##  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE
```

```
map_lgl(mtcars, \(x) any(x > 200))
```

```
##   mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
## FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

Takeaway: A predicate function answers a yes/no question, making `map_lgl()` a natural choice.

Example 15 --- Use `imap()` when you need column names as well as values

Dataset: `iris`.

Goal: Understand how `imap()` differs from `map()`.

Worked example

```
# imap() passes both the column itself (x) and its name (nm).
column_messages <- imap_chr(
  iris,
  \(x, nm) paste(nm, "contains", length(unique(x)), "distinct values")
)
column_messages
```

```
##                               Sepal.Length
## "Sepal.Length contains 35 distinct values"
##                                Sepal.Width
##  "Sepal.Width contains 23 distinct values"
##                               Petal.Length
## "Petal.Length contains 43 distinct values"
##                                Petal.Width
##  "Petal.Width contains 22 distinct values"
##                                    Species
##       "Species contains 3 distinct values"
```

Discuss: `map()` gets each column; `imap()` gets the column **and** its name (or position if the input has no names).

Your turn

Use `imap_chr()` on the numeric columns of `mtcars` to generate text such as `mpg: mean = 20.09` with the mean rounded to two decimal places.

Solution — Example 15

```
map_messages <- imap_chr(
  mtcars,
  \(x, nm) paste0(nm, ": mean = ", sprintf("%.2f", mean(x)))
)
map_messages
```

```
##                  mpg                  cyl                 disp
##   "mpg: mean = 20.09"    "cyl: mean = 6.19" "disp: mean = 230.72"
##                   hp                 drat                   wt
##   "hp: mean = 146.69"   "drat: mean = 3.60"     "wt: mean = 3.22"
##                 qsec                   vs                   am
##  "qsec: mean = 17.85"     "vs: mean = 0.44"     "am: mean = 0.41"
##                 gear                 carb
##   "gear: mean = 3.69"   "carb: mean = 2.81"
```

Takeaway: This is useful for automatic reports, validation messages, and readable summaries.

Example 16 --- Use `map_if()` to change only appropriate list elements

Dataset: `iris`.

Goal: Selectively modify elements while preserving others.

Worked example

```
# Center only numeric columns; leave the Species factor unchanged.
centered <- map_if(
  iris,
  is.numeric,
  \(x) x - mean(x)
)

class(centered)         # list
```

```
## [1] "list"
```

```
map_chr(centered, \(x) class(x)[1])
```

```
## Sepal.Length  Sepal.Width Petal.Length  Petal.Width      Species
##    "numeric"    "numeric"    "numeric"    "numeric"     "factor"
```

```
map_dbl(centered[1:4], mean)  # Approximately zero
```

```
##  Sepal.Length   Sepal.Width  Petal.Length   Petal.Width
## -3.666902e-16  9.169893e-17 -3.158931e-17 -3.359437e-17
```

```
# Convert the list back to a data frame if needed.
centered_df <- as_tibble(centered)
```

Discuss: `map_if()` is a conditional mapping tool. For typical data-frame column transformations, `mutate(across(where(is.numeric), ...))` is also a good approach.

Your turn

Create a data frame in which every numeric column of `iris` is divided by its standard deviation, but `Species` is left untouched. Verify that the result still contains 150 rows and five columns.

Solution — Example 16

```
scaled_columns <- map_if(iris, is.numeric, \(x) x / sd(x))
result <- as_tibble(scaled_columns)
dim(result)
```

```
## [1] 150   5
```

```
class(result$Species)
```

```
## [1] "factor"
```

```
# An equivalent dplyr approach:
result_dplyr <- iris %>%
  mutate(across(where(is.numeric), \(x) x / sd(x)))
```

Takeaway: `map_if()` selects elements according to a predicate and applies the function only to those elements.

Example 17 --- Use `map2()` for different rules for different groups

Dataset: `mtcars`.

Goal: Use one list of values together with a second vector of thresholds.

Worked example

```
hp_by_gear <- split(mtcars$hp, mtcars$gear)

# A different horsepower cutoff for each gear group.
cutoffs <- c(`3` = 150, `4` = 100, `5` = 200)

# Match cutoffs to the order of the list's group names.
cutoffs <- cutoffs[names(hp_by_gear)]

map2_int(
  hp_by_gear,
  cutoffs,
  \(horsepower, limit) sum(horsepower > limit)
)
```

```
##  3  4  5
## 10  5  2
```

Discuss: `map()` supplies one varying input, while `map2()` supplies two corresponding inputs. The first horsepower vector uses the first cutoff, and so on.

Your turn

For each gear group, compute the **fraction** of cars above its cutoff using `map2_dbl()`. Confirm the outputs are between 0 and 1.

Solution — Example 17

```
fractions <- map2_dbl(
  hp_by_gear,
  cutoffs,
  \(horsepower, limit) mean(horsepower > limit)
)
fractions
```

```
##         3         4         5
## 0.6666667 0.4166667 0.4000000
```

```
all(fractions >= 0 & fractions <= 1)
```

```
## [1] TRUE
```

Takeaway: A logical expression is converted to a fraction by `mean()` because `TRUE` counts as 1 and `FALSE` as 0.

Example 18 --- Use `pmap()` when a function needs several inputs

Dataset: `mtcars`.

Goal: Feed multiple columns into a function one row at a time.

Worked example

```
# Fit a model to the built-in mtcars dataset.
fit <- lm(mpg ~ wt + hp, data = mtcars)

# Use three observations as a demonstration.
new_cars <- mtcars[1:3, c("wt", "hp")]

# Each iteration receives wt and hp from one row.
predictions <- pmap_dbl(
  new_cars,
  \(wt, hp) {
    new_observation <- data.frame(wt = wt, hp = hp)
    as.numeric(predict(fit, newdata = new_observation))
  }
)
predictions
```

```
## [1] 23.57233 22.58348 25.27582
```

```
# Check against the usual vectorized prediction call.
predict(fit, newdata = new_cars)
```

```
##     Mazda RX4 Mazda RX4 Wag    Datsun 710
##      23.57233      22.58348      25.27582
```

Discuss: We normally prefer the vectorized `predict()` call for this specific task. `pmap()` is useful here to **illustrate** how to iterate across multiple inputs, especially when each row requires a more complicated custom operation.

Your turn

Use `pmap_dbl()` to predict `mpg` for the first **five** cars in `mtcars`, using the same model. Check that your results match `predict(fit, ...)`.

Solution — Example 18

```
five_cars <- mtcars[1:5, c("wt", "hp")]

five_predictions <- pmap_dbl(
  five_cars,
  \(wt, hp) as.numeric(predict(
    fit,
    newdata = data.frame(wt = wt, hp = hp)
  ))
)

five_predictions
```

```
## [1] 23.57233 22.58348 25.27582 21.26502 18.32727
```

```
all.equal(unname(five_predictions), unname(predict(fit, newdata = five_cars)))
```

```
## [1] TRUE
```

Takeaway: `pmap()` generalizes `map2()` to an arbitrary collection of input arguments.

Example 19 --- Fit a separate regression for each group

Dataset: `mtcars`.

Goal: Connect the earlier ideas of splitting and mapping to grouped statistical modeling.

Worked example

```
# Method A: split + map()
cars_by_cyl <- split(mtcars, mtcars$cyl)
models <- map(cars_by_cyl, \(d) lm(mpg ~ wt, data = d))
map_dbl(models, \(model) unname(coef(model)["wt"]))
```

```
##         4         6         8
## -5.647025 -2.780106 -2.192438
```

```
# Method B: group_modify() returns a combined data frame.
slopes <- mtcars %>%
  group_by(cyl) %>%
  group_modify(\(d, key) {
    fit_group <- lm(mpg ~ wt, data = d)
    tibble(slope = unname(coef(fit_group)["wt"]))
  })
slopes
```

```
## # A tibble: 3 × 2
## # Groups:   cyl [3]
##     cyl slope
##   <dbl> <dbl>
## 1     4 -5.65
## 2     6 -2.78
## 3     8 -2.19
```

Discuss: `group_modify()` passes the group's data as its first argument and its grouping key as its second; it combines the group-specific output data frames automatically.

Your turn

For every cylinder group, fit `lm(mpg ~ hp)` and return a combined data frame containing the cylinder count and the slope for `hp`.

Solution — Example 19

```
mtcars %>%
  group_by(cyl) %>%
  group_modify(\(d, key) {
    group_fit <- lm(mpg ~ hp, data = d)
    tibble(hp_slope = unname(coef(group_fit)["hp"]))
  })
```

```
## # A tibble: 3 × 2
## # Groups:   cyl [3]
##     cyl hp_slope
##   <dbl>    <dbl>
## 1     4 -0.113
## 2     6 -0.00761
## 3     8 -0.0142
```

Takeaway: This is useful when the same statistical model must be fitted independently to multiple groups.

Example 20 --- Use `walk()` for actions such as writing files

Dataset: `iris`.

Goal: Distinguish calculating a value from producing a side effect.

Worked example

```
# Create a temporary directory, rather than cluttering the project folder.
output_dir <- file.path(tempdir(), "data612_species")
dir.create(output_dir, showWarnings = FALSE)

species_data <- split(iris, iris$Species)

# walk2() pairs each group name with its corresponding data frame.
walk2(
  names(species_data),
  species_data,
  \(nm, d) write.csv(
    d,
    file = file.path(output_dir, paste0(nm, ".csv")),
    row.names = FALSE
  )
)

list.files(output_dir)
```

```
## [1] "setosa.csv"     "versicolor.csv" "virginica.csv"
```

Discuss: `walk2()` is for **doing something** for each pair of inputs. The desired result is the written CSV files, not a list of computed values. `walk()` is the one-input version.

Your turn

Split `mtcars` by `gear`. Create one CSV file for each gear count in a new temporary folder, with file names `gear_3.csv`, `gear_4.csv`, and `gear_5.csv`. Check that all three files exist.

Solution — Example 20

```
gear_dir <- file.path(tempdir(), "data612_gears")
dir.create(gear_dir, showWarnings = FALSE)

cars_by_gear <- split(mtcars, mtcars$gear)
walk2(
  names(cars_by_gear),
  cars_by_gear,
  \(gear_name, group_data) write.csv(
    group_data,
    file.path(gear_dir, paste0("gear_", gear_name, ".csv")),
    row.names = FALSE
  )
)

list.files(gear_dir)
```

```
## [1] "gear_3.csv" "gear_4.csv" "gear_5.csv"
```

```
length(list.files(gear_dir)) == 3
```

```
## [1] TRUE
```

Takeaway: Choose `walk()` or `walk2()` when the primary purpose is saving files, printing reports, or other side effects.

End-of-class concept check

Try to answer these without running R first:

1. What is the difference between `split(mtcars$hp, mtcars$gear)` and `split(mtcars, mtcars$gear)`?
2. Why does `map_dbl(x, quantile)` usually fail if `x` contains numeric vectors?
3. What two inputs does the function inside `imap()` receive?
4. How is `map2()` different from `pmap()`?
5. What is the difference between `map()` and `walk()`?
6. When should you use `group_modify()` rather than `group_map()`?

Instructor answer key

1. The first splits just horsepower into numeric vectors. The second splits every row and column of the data frame into data frames by gear.

2. With default probabilities, `quantile()` returns five numbers, but `map_dbl()` requires one double value from each element (and missing values can also cause an error).
3. The element and its name (or position, when unnamed).
4. `map2()` iterates over two varying inputs; `pmap()` can use many varying inputs.
5. `map()` collects outputs in a list; `walk()` is intended primarily for side effects and invisibly returns its input.
6. Use `group_modify()` when each group produces a data frame and you want those group results combined into one data frame.

Functions practiced in examples 11--20

`split()`, `map()`, `map_dbl()`, `map_int()`, `map_chr()`, `map_lgl()`, `map_if()`, `imap_chr()`, `map2_int()`, `map2_dbl()`, `pmap_dbl()`, `group_modify()`, and `walk2()`.

Installation (only if needed): `install.packages("tidyverse")`.